

Relatório de Desempenho: Simulação de Incêndio Florestal com OpenMP

[Nome dos Integrantes / Número USP]

Disciplina: Computação de Alto Desempenho

Universidade de São Paulo (USP)

11 de setembro de 2026

1 Descrição das Soluções

1.1 Solução Sequencial

A solução sequencial (`fire_seq.c`) simula a propagação de incêndio através de uma grade bidimensional de células ao longo de P passos discretos de tempo. A cada passo de tempo, a simulação itera de forma determinística sobre a malha de $L \times C$ células, avaliando o estado atual de cada célula em conjunto com os estados de seus vizinhos mais próximos e as direções do vento. A partir dessas avaliações, calculam-se as transições de estado para o próximo passo e acumula-se a contagem global dos dados estatísticos (células não combustíveis, intactas, em chamas, queimadas e sob contenção).

1.2 Solução Paralela e Estratégia de Paralelização

A versão paralela (`fire_omp.c`) utiliza a biblioteca **OpenMP** direcionada para arquiteturas de memória compartilhada, aplicando as seguintes estratégias de otimização:

- **Região Paralela Persistente:** Foi definida uma única região paralela (`#pragma omp parallel`) em torno do laço temporal principal (`while (passo < P)`). Esta abordagem elimina o overhead de criação e destruição repetitiva da equipe de *threads* a cada iteração de tempo.
- **Escopo Estrito de Variáveis (`default(none)`):** Para evitar condições de corrida (*race conditions*), o escopo de cada variável foi explicitado. As matrizes globais foram configuradas como `shared(...)`, enquanto contadores e iteradores locais foram definidos como privados.
- **Paralelização e Vetorização (`#pragma omp for simd`):** Os laços internos de atualização das células utilizam a instrução combinada `for simd`, permitindo a distribuição das iterações entre as *threads* e o uso de instruções vetoriais da CPU (AVX/SSE).

- **Redução de Variáveis (`reduction(+: ...)`):** Os contadores estatísticos locais (`l_nc`, `l_int`, `l_cha`, `l_que`, `l_cnt`) são consolidados via cláusula de redução ao final das iterações, garantindo atomicidade sem o custo computacional de seções críticas (`#pragma omp critical`).
- **Escalonamento Dinâmico/Guiado (`schedule(runtime)`):** O escalonamento é parametrizado em tempo de execução pela variável de ambiente `OMP_SCHEDULE`, permitindo flexibilidade na escolha do tamanho de bloco (*chunk size*).

2 Ambiente Experimental

Todos os experimentos foram conduzidos sob o mesmo ambiente de hardware e software:

- **Processador:** Intel® Core™ i7-4790 CPU @ 3.60GHz (frequência máxima de 4,00 GHz).
- **Arquitetura:** 1 *socket*, 4 núcleos físicos de processamento, 2 *threads* por núcleo (Hyper-Threading habilitado, totalizando 8 CPUs virtuais).
- **Memória Cache:** L1d de 128 KiB, L2 de 1 MiB e L3 compartilhada de 8 MiB.
- **Memória RAM:** 31 GiB de memória principal.
- **Sistema Operacional e Compilador:** Ubuntu Linux, gcc versão 5.3.1.
- **Flags de Compilação:** `gcc -Wall -Wextra -O3 -std=c99 -fopenmp`.
- **Parâmetros de Execução Paralela:** Execução fixada em 8 *threads* (número total de *threads* lógicas disponíveis no nó de teste do cluster Andromeda), avaliando as políticas `dynamic`, `512` e `guided`, `512`.

3 Validação da Versão Paralela

A validação da correção algorítmica da versão paralela foi realizada comparando suas saídas com as da versão sequencial para as três cargas de entrada padrão.

Tabela 1: Comparação de Checksums entre as Versões Sequencial e Paralela

Carga	Passos	Checksum Sequencial	Checksum Paralelo	Status
Pequena	80	12001375052313148138	12001375052313148138	Idêntico
Média	100	15472542045181629368	15472542045181629368	Idêntico
Grande	100	13879468400959847931	13879468400959847931	Idêntico

A correspondência exata de todos os estados finais e do *checksum* de 64 bits confirma a integridade e o determinismo da implementação em OpenMP.

4 Resultados de Desempenho e Métrica de Escala

As métricas de *Speedup* (S) e Eficiência (E) foram calculadas conforme as equações:

$$S = \frac{T_{\text{sequencial}}}{T_{\text{paralelo}}} \quad (1)$$

$$E = \frac{S}{P} \times 100\% \quad (2)$$

Tabela 2: Tempos de Execução, Speedup e Eficiência para 8 Threads

Carga	Versão / Política	Tempo (s)	Speedup (S)	Eficiência (E)
Pequena	Sequencial	0,2741	1,00×	100,00%
	Paralela (dynamic,512)	0,9912	0,28×	3,46%
	Paralela (guided,512)	0,2672	1,03×	12,82%
Média	Sequencial	3,1935	1,00×	100,00%
	Paralela (dynamic,512)	2,4090	1,33×	16,57%
	Paralela (guided,512)	2,4339	1,31×	16,40%
Grande	Sequencial	11,1908	1,00×	100,00%
	Paralela (dynamic,512)	8,4613	1,32×	16,53%
	Paralela (guided,512)	8,4300	1,33×	16,59%

4.1 Inclusão dos Gráficos

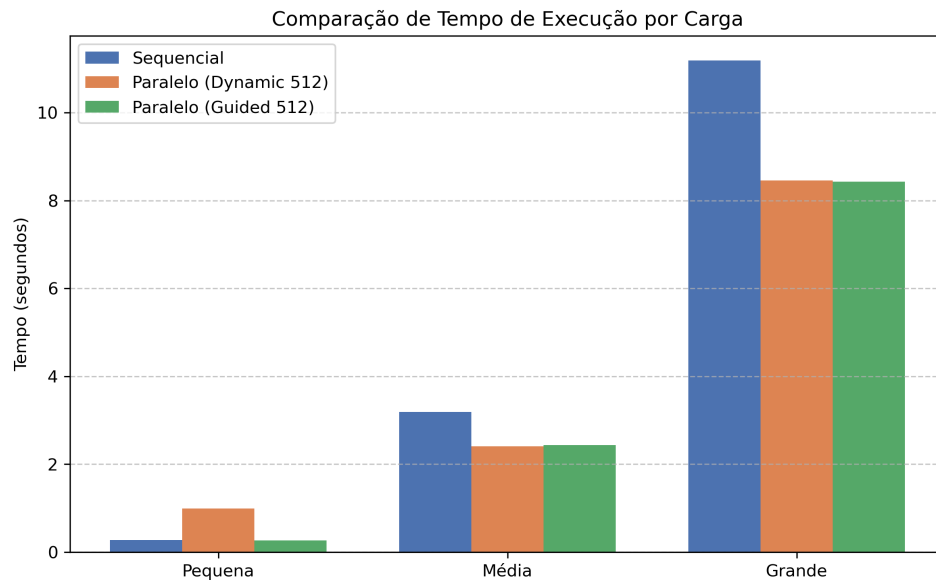


Figura 1: Comparativo dos Tempos de Execução entre as versões Sequencial e Paralelas.

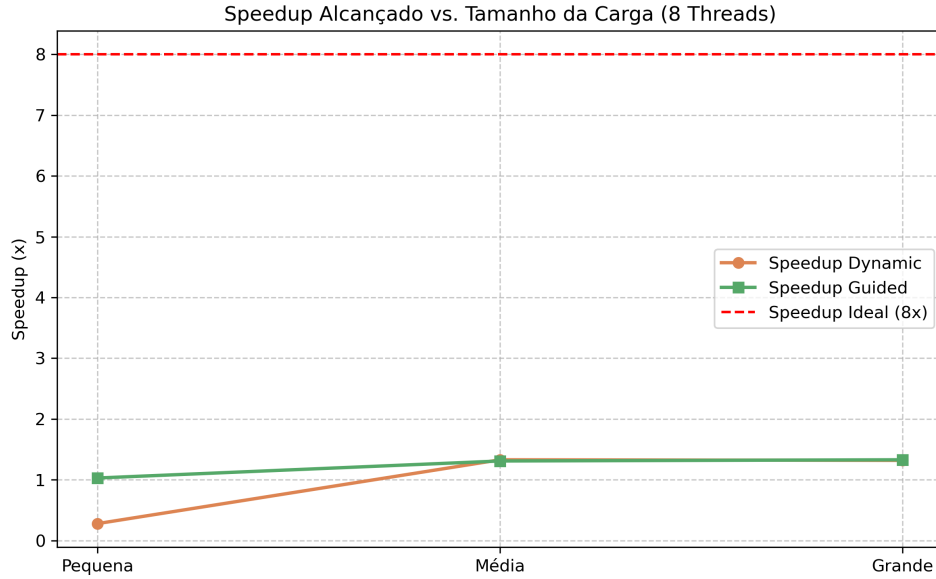


Figura 2: Speedup relativo alcançado com 8 threads nas cargas Pequena, Média e Grande.

5 Análise dos Resultados e Discussão

1. **Cargas Pequenas e Overhead de Gerenciamento:** Na carga pequena, a política *dynamic*,512 apresentou desaceleração ($0,28\times$), pois o tempo de overhead do OpenMP para controlar a fila de escalonamento superou o tempo de computação das células. A política *guided*,512 atenuou essa penalidade ($1,03\times$) ao alocar blocos inicialmente maiores.
2. **Gargalo de Banda de Memória (*Memory-Bound*):** Nas cargas Média e Grande, o *speedup* estabilizou em torno de $1,32\times$ a $1,33\times$. O fator limitante para a aceleração linear com 8 *threads* é a natureza da aplicação: o algoritmo realiza poucas operações aritméticas por cada acesso à memória RAM (leitura e escrita repetida em vetores extensos). As *threads* concorrentes saturam o barramento de memória da placa-mãe (*Bus Bottleneck*).
3. **Otimização do Compilador na Linha de Base Sequencial:** O compilador GCC com a flag `-O3` aplicou auto-vetorização eficiente na versão sequencial mono-thread. Isso elevou a taxa de desempenho do código sequencial base, tornando o ganho de escala relativo da versão paralela mais moderado, embora condizente com os limites físicos do hardware.